

Decision Trees

A **decision tree** is an alternative method of solving classification problems. They work by recursively splitting the inputs into purer and purer groups, such that at the end of the process, each leaf node is dominated by 1 group.

There are several reasons why they are important. Firstly, they are **highly interpretable**, you can follow the exact sequence of steps that resulted in a decision. Secondly, some concepts learned here appear across many other ML concepts.

Information Theory Foundations

We said that at each step, we want to partition our node (a subset of the training data), into two groups such that each group is **"purer"** or **"less mixed"** than we started. This begs the question, how do we measure this **purity**? We answer this using **information theory**.

Let's start with a simple example.

Consider a bag of **10 balls**, coloured red and blue. We're going to **pick one at random**, but before we do, we want to know, how **uncertain** am I about which colour I will get?

Why is this a relevant question? Well, if we are **very uncertain**, then we can genuinely make no reasonable guess as to which colour we will draw. Thus, there must be a **50/50** split between colours. Conversely, if we are **not very uncertain**, we can make an accurate prediction as to which colour we will get, thus, the bag must be **dominated by one colour**.

We call this uncertainty **Entropy**, and

intuitively we want to **decrease the entropy as much as possible with each step**, thus increasing the certainty that a randomly chosen data point from a leaf node belongs to a specific class. So how do we calculate the entropy, H ? We need a function that satisfies:

- $H = 0$ when all the balls are the same color. (There is zero uncertainty, we know exactly what color we'll pick)
- H is **maximised** when balls are split evenly across all colours. (Maximum uncertainty, we can make no good guess, as to the color.)
- H **increases smoothly** as the distribution becomes more uniform.

Let's consider the simplest case: a **single event** with probability p . How surprising is it when this event happens? (Surprise is kind of the same thing as uncertainty, but retrospective instead of prospective) We want a function $I(p)$ that satisfies:

- **Mono tonicity**. Rarer events are more surprising. If $p_1 < p_2$ then $I(p_1) > I(p_2)$
- **Certainty gives no information**. If $p = 1$, the event was guaranteed so $I(1) = 0$.
- **Independence is additive**. If two events are completely independent, knowing one tells you nothing about the other. Thus the total surprise of seeing both should be the sum of their individual surprises. $I(p_1 \cdot p_2) = I(p_1) + I(p_2)$

The only continuous function satisfying all three properties is

$$I(p) = -\log_2(p)$$

We now make a key observation:

Your **prospective uncertainty** before a draw is exactly equal to the **average retrospective surprise** you would experience if you made that draw many times.

$$H = E(I(y)) = \sum_k p_k I(p_k) \\ = - \sum_k p_k \log_2(p_k)$$

The expected value of $I(y)$ is the probability weighted average of the surprise for each class. Why do we need this weighting? Suppose we have three classes:

- **Cat**: $p = 0.98$, so $I = -\log_2(0.98) \approx 0.03$ bits - barely surprising

- **Dog**: $p = 0.01$, **Fish**: $p = 0.01$

So $I = -\log_2(0.01) \approx 6.64$ bits very surprising

If we just summed them up, we would get a high number, suggesting high uncertainty. But this node is actually nearly pure (98% cats). We should be almost certain about our choice. Dog and Fish are very surprising, but we almost never draw them, the high surprise doesn't really matter in practice. Thus we weight by p , surprise that are more likely to happen bear greater importance on the overall uncertainty of the configuration.

Back to the Tree

So we have our measure of entropy

$$H = - \sum_k p_k \log_2(p_k)$$

And we want to choose splits to drive down the entropy of the leaf nodes. We make the split based on a **feature threshold**. (e.g. weight ≤ 10 kg). Each group goes into a child node, each with their own entropy. The question is: **did the split make things better?** We use a quantity called **information gain** to check:

$$IG = H(\text{parent}) - [\text{weighted average } H \text{ of children}]$$

We weight child entropies by the fraction of examples going to each child. This weighting ensures that a split that creates one tiny pure child and one large mixed child isn't credited as a good split. The large mixed child shows that most of the uncertainty is still unresolved. A good split must reduce entropy across both children in proportion to their size. A child containing 99% of the examples matters for more than one containing 1%.

So the full process is this:

- At every node, we try **every possible feature** and **every possible threshold**.

For the candidate split we compute the information gain. We pick the one with the largest information gain, make the split and recurse on each child.

- We stop when a node is already pure ($H=0$), too small to split, or we've hit a predefined depth limit.

At the end, we set the leaf nodes to the majority class, such that each unseen data point filters down to a class label.

How do we pick the thresholds?

For a continuous feature, there are technically infinitely many thresholds.

However, if each feature holds genuine bearing on the class, there will be some threshold at which the input will belong to class B instead of A.

For example, if predicting pets, and considering the weight feature, there is some weight x where every weight greater than x is a dog, and every weight $\leq x$ is a cat.

Since we don't know where this threshold is, we sort the features, and take the midpoints of every adjacent pair as our candidate thresholds.

Note that this approach is very computationally heavy, and is a greedy algorithm, so it does not necessarily result in the most optimal tree.

Overfitting and Regularisation

If we let the algorithm run for too long, it will eventually split the training data so much that each leaf node contains only a single (or small group of) training example(s). At this point it has **perfectly memorised** the training data - a disaster for generalisation.

The issue is that in real life, all measurements contain some degree of noise. A fully grown tree treats noise the same as a genuine signal. The tree ends up creating splits that perfectly classify the noisy example in the training, but any new noisy examples will certainly be misclassified.

Thus, trees are very **high variance**, they are very sensitive to their inputs. How do we stop this?

Pre - Pruning

Pre-pruning uses some **criteria** to stop the tree from **growing too large / deep**. A shallow tree makes a few broad, reliable splits on genuinely informative features. Many fine-grained splits are eventually just fitting noise.

Common stopping criteria include:

- **Maximum depth**

Do not split any node at depth $> \text{max_depth}$.
Directly limits model complexity.

- **Minimum samples to split**

Only split a node if it contains at least k examples.

Prevents splits on tiny, noisy subsets.

- **Minimum Samples Per leaf**

Only allow a split if **both** resulting children have at least k examples.
Prevents creating leaves with one or two noisy samples.

- **Minimum Information gain**

Only split if the best split achieves an information gain over some chosen threshold.

Post - Pruning

Post Pruning first builds the full tree, then removes branches that do not improve generalisation. The standard approach is **cost-complexity pruning**, also known as **weakest link pruning**. We want to find the simplest tree that fits the data reasonably well, in other words, we're trying to penalise **complexity**.

For a tree T , we define $R(T)$ to be the training error of the tree, the fraction of misclassified training examples. We then define a complexity penalty:

$$R_\alpha(T) = R(T) + \alpha \cdot |leaves(T)|$$

Where $|leaves(T)|$ is the number of leaf nodes, a direct measure of tree complexity. $\alpha \geq 0$ controls how harshly we penalize complexity.

For a given α , the algorithm considers every node and asks: is it worth keeping this subtree or should I collapse it into a single leaf? Collapsing the subtree increases the training error by some amount ΔR but removes some number of leaves $\Delta leaves$. The trade off is worth making when the ratio:

$$\frac{\Delta R}{\Delta leaves} \leq \alpha$$

when we lose many leaves for a small increase in error. This ratio is called the effective α of that subtree. The algorithm prunes the subtree with the smallest effective α first, then recomputes and repeats, until no more pruning is beneficial.

Post pruning tends to produce better trees than pre-pruning, but it is more computationally expensive.

Regression Trees

Everything above was relevant to classification, but trees can also be used for regression. The structure is essentially the same, with two key changes.

Of course, we no longer have class

class labels, so we need to change how we define impurity and how we make predictions.

Impurity

How do we measure how "mixed" a node is when the outputs are continuous numbers rather than categories?

For regression the analogous quantity to entropy is variance.

$$Var(S) = \frac{1}{n_s} \sum_{i=1}^{n_s} (y_i - \bar{y})^2$$

Where $\bar{y}$ is the mean of y across all examples in node S . High variance means the target values are spread out, low variance means they are all clustered around the mean. So now, we use variance reduction instead of information gain as our splitting criterion.

$$VR = Var(S) - \left[\frac{|S_L|}{|S|} \cdot Var(S_L) + \frac{|S_R|}{|S|} \cdot Var(S_R) \right]$$

The difference between the variance of the parent and weighted sum of the variances of the children. We compute this for every feature and threshold and pick the split with the highest VR.

Predictions

Once a new example reaches a leaf, the prediction is simply the mean of the target values of all training examples that ended up in that leaf.

It's important to know what the model looks like. The regression tree partitions the input space into rectangular

regions that each predict a constant value. The result is a piecewise constant function, it steps abruptly between values at the split boundaries rather than varying smoothly.

Importantly, this means regression trees cannot extrapolate beyond the range of training data.

Feature importance

A useful property of decision trees is that they provide a natural measure of feature importance. For each feature j , its importance is the total information gain (weighted by node size) across all splits that used feature j .

$$\text{Importance}(j) = \sum \frac{|S_{\text{node}}|}{n} \cdot \text{IG}(S_{\text{node}}, \text{splits } j)$$

Features that produce high information gain are assigned high importance. Features that are never used get zero importance.

This gives decision trees a built-in form of feature selection, after training, you can rank features by importance and discard those with near-zero importance. Especially useful in high-dimensional settings.